Лекция 5. Градиенты, границы и преобразование Хафа

Производные изображения, Собель, производная гауссиана, LoG, детектор Кэнни, Хаф для прямых и окружностей, выравнивание документа

Александр Савельев · Курс «Анализ изображений», 3 курс бакалавриата

Повторение: что нужно из лекций 2–4

- **Корреляция** $(I \star h)[m,n] = \sum_{u,v} I[m+u, n+v]\, h[u,v]$ — взвешенная сумма соседей; ядро h задаёт, **что** мы ищем (лекция 3).
- **Собель** уже встречался как пример разделимого ядра: $[1,2,1]^\top \times [-1,0,1]$. Сегодня — **зачем** он нужен.
- **Гаусс** G_σ гасит шум; размер ядра — правило 3σ (лекция 3).
- **Частоты** (лекция 4): шум и края — высокие частоты; фильтр, усиливающий высокие частоты, усиливает и шум.
- **Бинаризация** Оцу (лекция 2) понадобится в конце.

Сегодня: от пикселей к контурам

До сих пор фильтр давал **новую картинку**. Сегодня из картинки получим **структуру**: тонкие линии границ, а из них — прямые и окружности с параметрами.

Главный вопрос лекции

Как найти границы объектов так, чтобы они были **тонкими, непрерывными** и не реагировали на шум?

План лекции

1. **Что такое граница** — типы перепадов и откуда они берутся — 6 мин
2. **Производная изображения и градиент** — разности, шум, Собель — 18 мин
3. **Производная гауссиана** — правильный оператор и масштаб границы — 10 мин
4. **Вторые производные** — лапласиан, LoG, DoG, Марр–Хилдрет — 10 мин
5. **Детектор Кэнни** — пять шагов и зачем нужен каждый — 22 мин
6. **Преобразование Хафа** — прямые, окружности, обобщения — 19 мин
7. **Пример: выравнивание скана документа** — 5 мин

Сквозная линия

Граница — место, где яркость **быстро меняется**, то есть где велика производная. Всё остальное — как посчитать производную **устойчиво к шуму**, как сделать границу **тонкой и связной** и как собрать из её пикселей **геометрические фигуры**.

Часть 1

Что такое граница

О чём эта часть

Определение

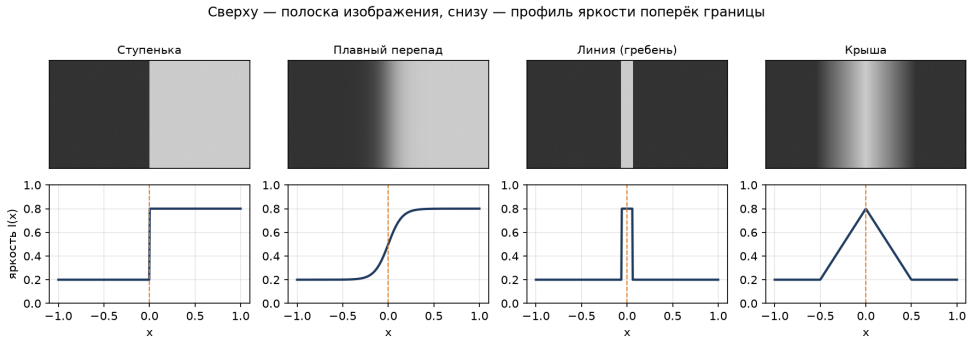
Граница (край, edge) — линия на изображении, поперёк которой яркость (или цвет) **резко меняется**, а вдоль — почти нет.

Детектор границ — алгоритм, который по изображению возвращает **карту границ**: для каждого пикселя «граница / не граница», иногда с силой и направлением.

Что покажем

- какие бывают перепады яркости;
- какие физические причины их создают;
- почему граница — это место большой производной.

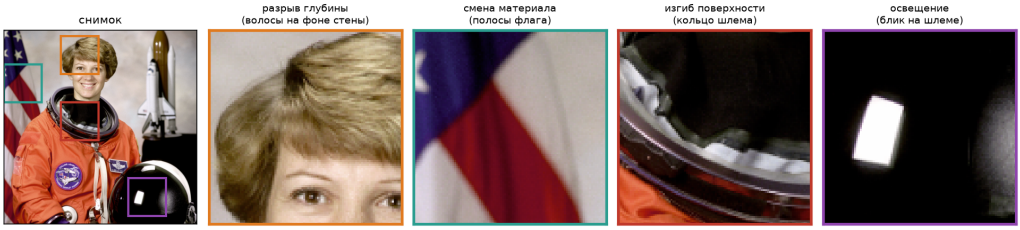
Типы границ



Ступенька — идеальный скачок; в реальности объектив и сенсор его размывают: получается **плавный перепад** шириной в несколько пикселей. Это самый частый случай.

Линия (гребень) — тонкий объект: провод, шов, штрих буквы — **два края рядом**. **Крыша** — излом без скачка (ребро предмета при боковом свете). Детекторы ступенек находят на них два края или ни одного.

Откуда берутся границы



Перепад яркости появляется, когда меняется хотя бы один множитель модели $I = \rho \cdot E$ из лекции 1: **отражательная способность** ρ (краска, материал), **освещённость** E (тень, блик) или **геометрия** — разрыв глубины и изгиб поверхности меняют, какая точка и под каким углом видна.

Важно: детектор видит только перепады яркости и **не знает их причину**. Граница тени и граница объекта для него одинаковы. Отличить их — задача следующих уровней анализа (сегментация, лекции 8–9).

Граница = большая производная

Идея. На ровном участке яркость почти не меняется, на границе — меняется быстро. «Скорость изменения» — это **производная**:

$$I'(x) = \lim_{h \to 0} \frac{I(x+h) - I(x)}{h}$$

- $I(x)$ — яркость вдоль строки как функция координаты x;
- h — маленький шаг; $I'(x)$ — на сколько вырастет яркость при шаге вправо на единицу.

Граница — там, где $|I'(x)|$ **максимален**. Знак показывает, светлеет или темнеет.

Три вопроса на всю лекцию

1. Как взять производную, если I задана только в пикселях, а h не бывает меньше 1?
2. Что делать с шумом: он тоже даёт быстрые изменения?
3. Как из «производная большая» получить **тонкую линию**, а из линий — **прямые и окружности**?

Часть 2

Производная изображения и градиент

О чём эта часть

Определение

Конечная разность — замена производной разностью соседних пикселей: $I'(x) \approx I[n+1] - I[n]$.

Градиент ∇I — вектор из двух частных производных: по горизонтали и по вертикали. Показывает, **куда** яркость растёт быстрее всего и **насколько** быстро.

Что покажем

- разности вперёд и центральные, их точность;
- градиент: модуль и направление;
- почему дифференцирование усиливает шум;
- операторы Робертса, Превитта, Собеля.

Конечные разности

$$\underbrace{I[n+1] - I[n]}_{\text{вперёд}} = I' + \tfrac{1}{2} I''\, h + \ldots, \qquad \underbrace{\tfrac{1}{2}\big(I[n+1] - I[n-1]\big)}_{\text{центральная}} = I' + \tfrac{1}{6} I'''\, h^2 + \ldots$$

- $I[n]$ — яркость пикселя n; шаг сетки $h = 1$ пиксель; I', I'', I''' — производные непрерывной яркости в точке n;
- правые части — ряд Тейлора: $I(x \pm h) = I \pm hI' + \frac{h^2}{2} I'' \pm \frac{h^3}{6} I''' + \ldots$; всё после I' — **ошибка** формулы.

Разность вперёд

Ядро $[-1,\ 1]$. Ошибка $\sim h$: на параболе ошибается. Ответ относится к точке $n + \frac{1}{2}$, а не к пикселю n: результат **сдвинут на полпикселя**.

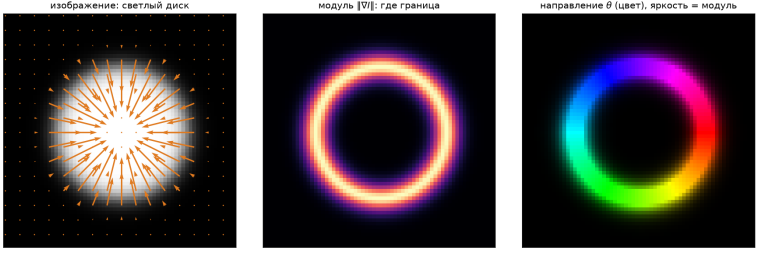
Центральная разность

Ядро $\frac{1}{2}[-1,\ 0,\ 1]$. Ошибка $\sim h^2$: точна для любой параболы. Симметрична — ответ ровно в пикселе n. **Её и используют** во всех операторах ниже.

Градиент: модуль и направление

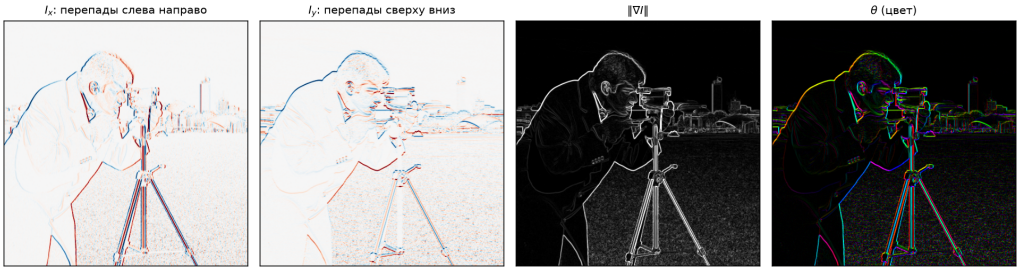
$$\nabla I = \big(I_x,\ I_y\big), \qquad \|\nabla I\| = \sqrt{I_x^2 + I_y^2}, \qquad \theta = \operatorname{atan2}(I_y,\ I_x)$$

- $I_x = \partial I/\partial x$ — производная вдоль строки (вправо), $I_y = \partial I/\partial y$ — вдоль столбца (**вниз**: ось y изображения смотрит вниз);
- $\|\nabla I\|$ — **сила границы**: насколько быстро меняется яркость в самом «крутом» направлении;
- θ — **направление роста** яркости (от тёмного к светлому); сама граница идёт **перпендикулярно** θ.



На светлом диске стрелки градиента смотрят к центру (туда светлее), модуль велик только на краю, направление поворачивается по кругу — цвет обходит всю палитру.

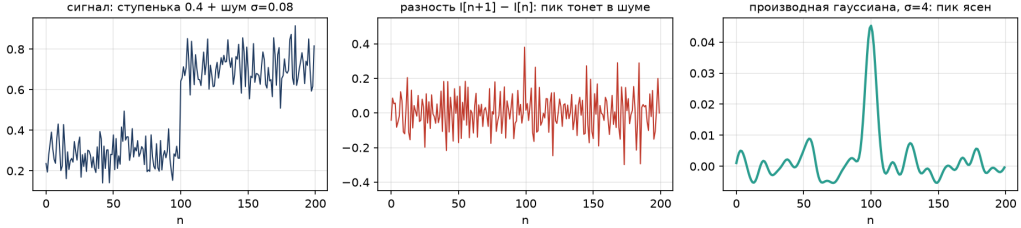
Градиент на фотографии



I_x подсвечивает **вертикальные** границы (штатив, край пальто): перепад идёт слева направо. I_y — **горизонтальные** (линия горизонта, плечи). Красное — светлеет, синее — темнеет.

$\|\nabla I\|$ уже похож на «рисунок контуров», но линии **толстые**, а на траве — россыпь слабых откликов. Цвет θ на одной границе постоянный: направление — полезный признак (понадобится в Кэнни и Хафе).

Проблема: производная усиливает шум



Опыт. Ступенька высотой 0.4 плюс шум $\sigma = 0.08$ — глазом ступенька видна отлично. Но разность соседей превращает каждый случайный «зубец» в пик: шум в разности (≈ 0.10) почти такой же, как пик ступеньки (≈ 0.4), и **случайные пики достигают его высоты**.

$$\mathrm{Var}\left(I[n{+}1] - I[n]\right) = 2\sigma^2$$

- шум в соседних пикселях независим, дисперсии **складываются**: шум разности в $\sqrt{2}$ раз больше шума пикселя;
- а полезный сигнал разности — лишь перепад **за один пиксель**, обычно малая доля всей ступеньки.

То же на изображении

снимок + шум σ=0.05



|разности| без сглаживания

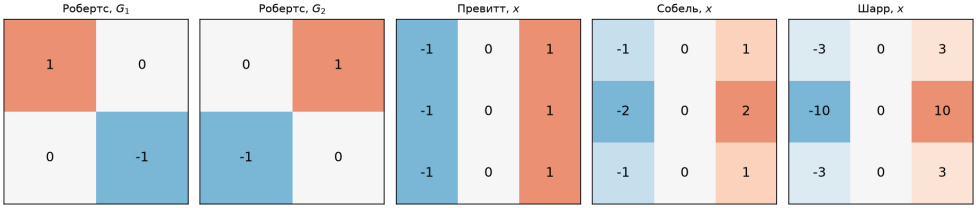


производная гауссиана, σ=2



Шум $\sigma = 0.05$ почти не мешает смотреть на снимок, но модуль разностей без сглаживания — **сплошная «рябь»**, контуры в ней теряются. После сглаживания Гауссом $\sigma = 2$ (а точнее — производной гауссиана, часть 3) остаются настоящие границы. Вывод: **сначала сгладить, потом дифференцировать**.

Робертс, Превитт, Собель, Шарр



- **Робертс** (1963): разности **по диагоналям** в окне 2×2. Очень быстрый, без сглаживания — самый шумный.
- **Превитт** (1970): центральная разность по x, **усреднённая** по трём строкам.
- **Собель** (1968): то же, но средняя строка с весом 2 — сглаживание ближе к Гауссу.
- **Шарр** (2000): веса $3, 10, 3$ подобраны так, чтобы ответ **меньше зависел от направления** границы.
- Ядро по y — транспонированное ядро по x.

Собель = сглаживание × разность

$$S_x = \frac{1}{8}\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} = \underbrace{\frac{1}{4}\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}}_{\text{Гаусс по } y} \cdot \underbrace{\frac{1}{2}\begin{pmatrix} -1 & 0 & 1 \end{pmatrix}}_{\text{производная по } x}, \qquad S_y = S_x^{\top}$$

- $\frac{1}{4}[1, 2, 1]$ — биномиальное ядро, маленький Гаусс ($\sigma \approx 0.7$): **сглаживает вдоль** границы;
- $\frac{1}{2}[-1, 0, 1]$ — центральная разность: **дифференцирует поперёк** границы;
- множитель $\frac{1}{8}$ делает ответ на наклон «1 уровень на пиксель» равным ровно 1 (без него — в 8 раз больше, так считает OpenCV).

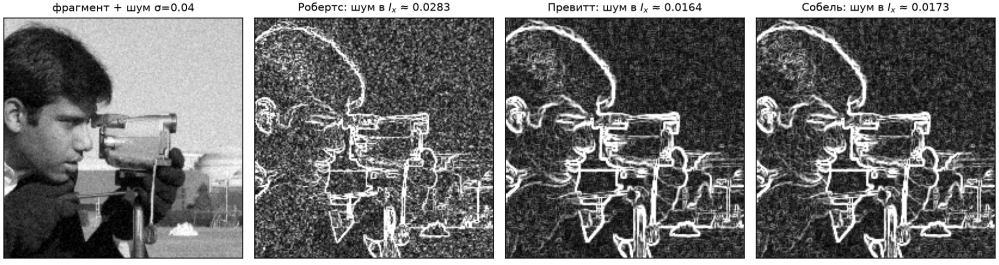
Почему сглаживать именно вдоль

Вдоль границы яркость почти не меняется — усреднение по этому направлению **гасит шум, не размывая край**.

Стоимость

Разделимость (лекция 3): две 1D-свёртки — 6 умножений вместо 9 на пиксель на каждую производную.

Сколько шума пропускает ядро



$$\operatorname{Var}\left(h \star n\right) = \sigma^2 \sum_{u,v} h[u,v]^2$$

- n — независимый шум с дисперсией σ^2; h — ядро;
- каждый вес вносит $h^2\sigma^2$, дисперсии складываются.

Для ядер, нормированных на единичный наклон: Превитт $\sqrt{6/36}\,\sigma \approx 0.41\sigma$, Собель $\sqrt{12/64}\,\sigma \approx 0.43\sigma$, центральная разность 0.71σ, разность вперёд 1.41σ. **Чем больше пикселей усредняется, тем меньше шум** — отсюда идея взять большое гладкое ядро.

Листинг: градиент в NumPy

```
import numpy as np
from scipy import ndimage as ndi

kx = np.array([[-1, 0, 1],
               [-2, 0, 2],
               [-1, 0, 1]]) / 8   # Собель
Ix = ndi.correlate(I, kx)      # по x
Iy = ndi.correlate(I, kx.T)    # по y
mag = np.hypot(Ix, Iy)         # модуль
theta = np.arctan2(Iy, Ix)     # -pi..pi

# производная гауссиана (часть 3)
s = 2.0
Ix = ndi.gaussian_filter(I, s, order=(0, 1))
Iy = ndi.gaussian_filter(I, s, order=(1, 0))
```

- `correlate`, а не `convolve`: свёртка переворачивает ядро и меняет **знак** производной (лекция 3).
- `I` — `float`: в `uint8` отрицательные I_x превратятся в большие положительные.
- `np.hypot` $= \sqrt{a^2 + b^2}$ без переполнения.
- `order=(0, 1)`: порядок производной по осям (строки, столбцы) — первая производная по столбцам, то есть по x.
- OpenCV: `cv2.Sobel(I, cv2.CV_32F, 1, 0)` — **без** множителя $\frac{1}{8}$.

Часть 3

Производная гауссиана

О чём эта часть

Определение

Производная гауссиана (DoG-фильтр, ∂G_σ) — ядро, равное производной гауссова ядра: одной свёрткой и сглаживает, и дифференцирует.

Масштаб границы — ширина перепада в пикселях; параметр σ задаёт, границы **какого масштаба** мы ищем.

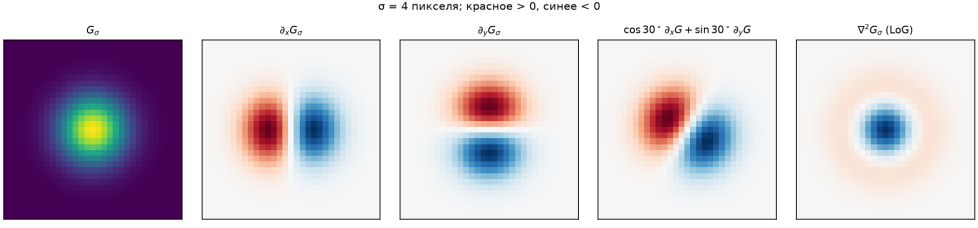
Что покажем

- почему «сгладить, потом продифференцировать» = одна свёртка;
- как выглядят ядра и как получить производную в любом направлении;
- компромисс выбора σ.

Сглаживание и производная сразу

$$\frac{\partial}{\partial x}\left(G_\sigma * I\right) = \left(\frac{\partial G_\sigma}{\partial x}\right) * I, \qquad \frac{\partial G_\sigma}{\partial x} = -\frac{x}{\sigma^2}\, G_\sigma(x,y)$$

- $G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/(2\sigma^2)}$ — гауссово ядро; $*$ — свёртка;
- левое равенство: дифференцирование — тоже свёртка (с ядром разности), а свёртки **можно переставлять** (ассоциативность, лекция 3): $d * (G * I) = (d * G) * I$;
- правое — просто производная экспоненты: множитель $-x/\sigma^2$.



Ядро $\partial_x G$: **минус слева, плюс справа** — «гладкий Собель» любого размера. Разделимо: $\partial_x G_\sigma = g'_\sigma(x) \cdot g_\sigma(y)$.

Производная в любом направлении

$$\partial_{\mathbf{u}} G_\sigma = \cos\alpha\, \partial_x G_\sigma + \sin\alpha\, \partial_y G_\sigma$$

- $\mathbf{u} = (\cos\alpha, \sin\alpha)$ — единичный вектор направления;
- $\partial_{\mathbf{u}}$ — скорость изменения вдоль $\mathbf{u}$; это скалярное произведение $\nabla I \cdot \mathbf{u}$.

Две свёртки (∂_x и ∂_y) дают производную **в любом** направлении без новых свёрток — такие фильтры называют **управляемыми** (steerable, Freeman, Adelson, 1991). Пример — четвёртое ядро на прошлом слайде ($\alpha = 30°$).

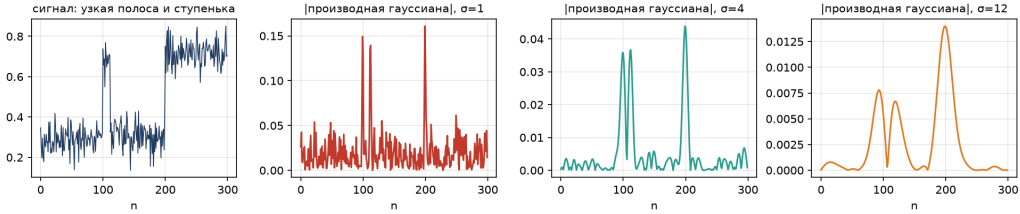
Почему градиент — «самое крутое» направление

$\nabla I \cdot \mathbf{u} = \|\nabla I\| \cos\beta$, где β — угол между $\mathbf{u}$ и ∇I.

- максимум при $\beta = 0$: вдоль градиента, значение $\|\nabla I\|$;
- ноль при $\beta = 90°$: **вдоль границы** яркость не меняется.

Поэтому модуль градиента — сила границы, а сама граница перпендикулярна ∇I.

Выбор σ: устойчивость против локализации



Сигнал: узкая светлая полоса (12 пикселей) и ступенька, шум $\sigma_n = 0.06$.

$\sigma = 1$: пики на месте, но шумовые пики почти такой же высоты — **ложные границы**.

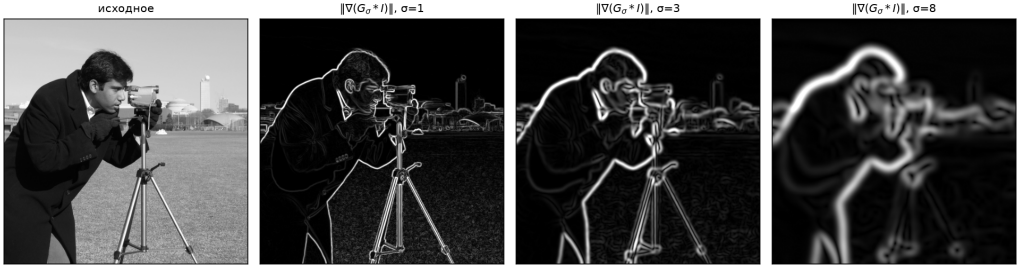
$\sigma = 4$: шум погашен, три пика чёткие — **хороший выбор**.

$\sigma = 12$: два края полосы **слились** в один горб, пики сдвинулись.

Компромисс

Больше σ — меньше шума (усредняем $\sim \sigma^2$ пикселей), но хуже **локализация**: пик шире, близкие края сливаются, углы скругляются. Нет одного «правильного» σ: он задаёт **масштаб** искомых границ.

Масштаб границ на фотографии



$\sigma = 1$: каждая травинка, буквы на здании, мелкая текстура пальто. $\sigma = 3$: силуэт, штатив, горизонт — **контуры объектов**. $\sigma = 8$: только крупные перепады, линии толстые и смещены. Практика: для «рисунка объектов» берут $\sigma \approx 1.5$–3 на снимках ~ 500 пикселей; несколько σ сразу — это масштабное пространство (лекция 6).

Часть 4

Вторые производные

О чём эта часть

Определение

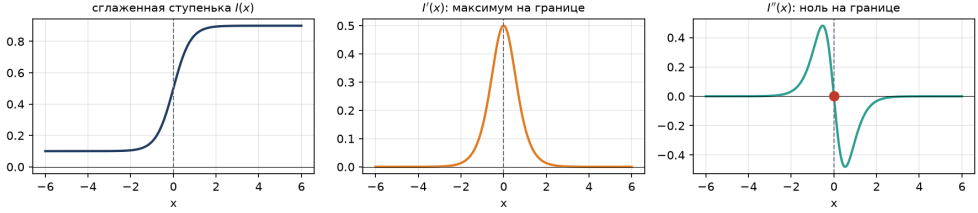
Лапласиан $\nabla^2 I = I_{xx} + I_{yy}$ — сумма вторых производных по x и по y; не зависит от поворота изображения.

Пересечение нуля (zero crossing) — место, где функция меняет знак. На границе первая производная максимальна, значит, вторая **проходит через ноль**.

Что покажем

- почему граница — ноль второй производной;
- лапласиан и его ядра;
- LoG и его быстрое приближение DoG;
- детектор Марра–Хилдрет и его слабости.

Граница — ноль второй производной



В точке максимума I' её производная I'' равна нулю и **меняет знак**: до границы $+$, после $-$ (или наоборот). Найти ноль проще, чем максимум: не нужно сравнивать с соседями и подбирать порог «насколько максимум».

Но: $I'' = 0$ и на ровном участке, и в точке перегиба слабой волны шума. Поэтому учитывают только нули с **большим перепадом** знака — с крутым наклоном I'' вокруг нуля.

Лапласиан и его дискретные ядра

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

- $\partial^2 I / \partial x^2$ — вторая производная вдоль строки; дискретно $I[n{-}1] - 2I[n] + I[n{+}1]$, ядро $[1, -2, 1]$;
- сумма по x и y — одно число на пиксель, **без направления**.

Лапласиан **изотропен**: при повороте картинки он не меняется (а I_x, I_y меняются). Поэтому хватает одного фильтра вместо двух.

Ядра 3×3

$$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix} \qquad \begin{pmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

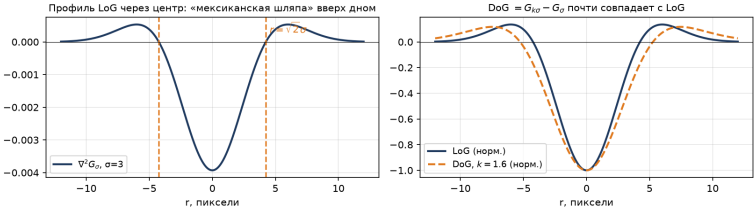
Слева — сумма $[1, -2, 1]$ по строке и столбцу (4 соседа). Справа — 8 соседей, ближе к изотропному. Сумма весов $= 0$: на ровном участке ответ 0.

Вторая производная шумит ещё сильнее первой: усиление $\sim (2\pi f)^2$. Без сглаживания лапласиан почти бесполезен.

LoG и его приближение DoG

$$\nabla^2 G_\sigma = \frac{r^2 - 2\sigma^2}{\sigma^4}\, G_\sigma, \qquad G_{k\sigma} - G_\sigma \approx (k-1)\,\sigma^2\,\nabla^2 G_\sigma$$

- $r^2 = x^2 + y^2$ — квадрат расстояния до центра ядра; ядро отрицательно при $r < \sqrt{2}\sigma$ и положительно снаружи;
- вывод LoG: $\partial_{xx} G = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G$, плюс то же по y;
- **DoG** — разность двух гауссиан с σ и $k\sigma$; приближение следует из $\frac{\partial G_\sigma}{\partial \sigma} = \sigma\,\nabla^2 G_\sigma$ (уравнение теплопроводности): $G_{k\sigma} - G_\sigma \approx (k\sigma - \sigma)\cdot \sigma \nabla^2 G_\sigma$.



При $k = 1.6$ (совет Марра и Хилдрет) DoG почти не отличим от LoG. DoG дешевле: два гауссовых сглаживания (разделимых) и разность. Это будет основой SIFT (лекция 6).

Детектор Марра–Хилдрет (1980)

Алгоритм

Вход: I, масштаб σ, порог t.
Выход: карта границ E.

1. $L \leftarrow \nabla^2 G_\sigma * I$ (или DoG)
2. для каждого пикселя p и соседа q (справа, снизу, по диагоналям):

   если $\operatorname{sign} L[p] \neq \operatorname{sign} L[q]$ и $|L[p] - L[q]| > t$ — $E[p] \leftarrow 1$

Нули LoG образуют **замкнутые** кривые: функция не может сменить знак «в никуда». Отсюда и плюс, и минус метода.



Без порога t — «спагетти»: нули есть везде, где слабый шум меняет знак L. С порогом остаются заметные контуры. Слабости: **скругляет углы**, даёт ложные границы на плавных перепадах, ответ сильно зависит от σ. Кэнни (дальше) решает эти задачи лучше.

Часть 5

Детектор Кэнни

О чём эта часть

Определение

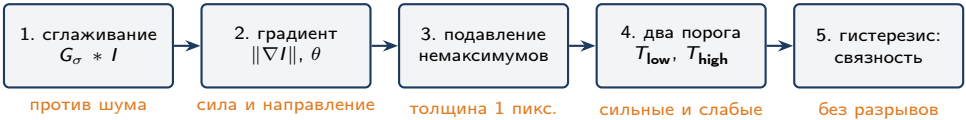
Детектор Кэнни (J. Canny, 1986) — алгоритм выделения границ из пяти шагов, выведенный из трёх требований к «хорошему» детектору:

1) **хорошее обнаружение** — мало пропусков и ложных срабатываний;
2) **хорошая локализация** — граница там, где она на самом деле;
3) **один отклик** на одну границу — линия толщиной 1 пиксель.

Что покажем

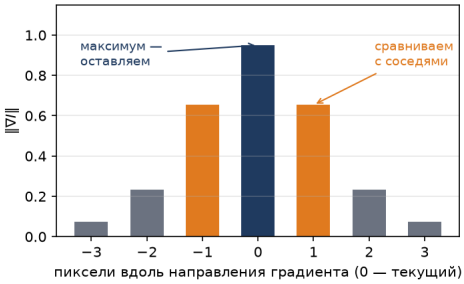
- пять шагов и зачем нужен каждый;
- подавление немаксимумов и гистерезис подробно;
- что ломается без каждого шага;
- как выбрать σ и пороги;
- субпиксельное уточнение.

Кэнни: пять шагов



Шаги 1–2 — производная гауссиана из части 3 (на практике Гаусс, затем Собель). Новое у Кэнни — шаги 3–5: они превращают «толстую» карту силы границ в **тонкие связные линии**. Кэнни показал, что для ступеньки с шумом оптимальный по трём критериям фильтр очень близок к ∂G_σ.

Шаг 3: NMS — идея



Проблема. После сглаживания модуль градиента поперёк границы — «холм» шириной 3–7 пикселей. Порог по модулю оставит полосу, а нам нужна **линия**.

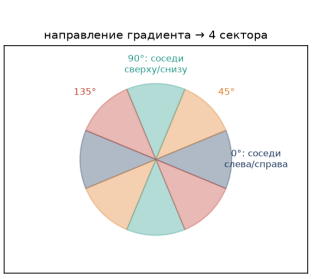
Идея. Граница — **вершина** холма. Пиксель оставляем, только если его модуль не меньше, чем у двух соседей **вдоль направления градиента** (поперёк границы):

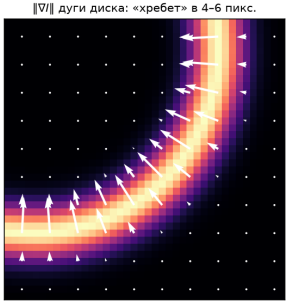
$$\|\nabla I\|(p) \geq \|\nabla I\|(p \pm \mathrm{d})$$

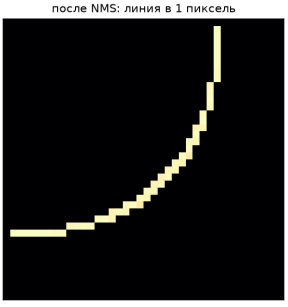
- p — текущий пиксель; d — шаг на соседний пиксель в направлении $\theta(p)$;
- иначе пиксель обнуляется: он на склоне, а не на вершине.

Вдоль границы соседей **не сравниваем** — там все одинаково большие, иначе линия разорвалась бы.

Шаг 3: NMS на пиксельной сетке







Квантование направления. Соседей у пикселя 8, поэтому θ округляют до одного из **4 направлений** $0°, 45°, 90°, 135°$ (противоположные направления дают ту же пару соседей). Сектор шириной $45°$ вокруг каждого.

Пример: $\theta = 80°$ — сектор $90°$, сравниваем с соседями **сверху и снизу**. Точнее — интерполировать модуль между двумя соседями в точке $p \pm (\cos\theta, \sin\theta)$; так делают `skimage` и многие реализации.

Шаги 4–5: двойной порог и гистерезис

Алгоритм

Вход: карта N после NMS, пороги $T_{\text{low}} < T_{\text{high}}$.

1. сильные: $S = \{p : N[p] \geq T_{\text{high}}\}$
2. слабые: $W = \{p : T_{\text{low}} \leq N[p] < T_{\text{high}}\}$
3. слабый пиксель оставляем, если он **связан цепочкой** слабых пикселей (8-соседство) с каким-нибудь сильным
4. остальное — не граница

Реализация — обход в ширину от сильных пикселей или разметка связных компонент $S \cup W$ с проверкой «есть ли в компоненте сильный».



Зачем два порога. Один высокий — граница рвётся там, где контраст ненадолго падает. Один низкий — проходят шумовые штрихи. Гистерезис: **начать** линию можно только с уверенного места, **продолжить** — и по слабому.

Что ломается без каждого шага



Один и тот же снимок с шумом $\sigma = 0.03$, одинаковые пороги.

Без сглаживания — шум даёт тысячи коротких «границ» на траве и небе.

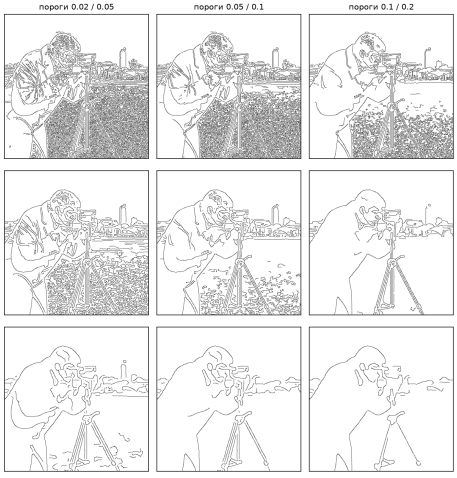
Без NMS — полосы толщиной в несколько пикселей: положение границы не определено.

Один низкий порог — проходят мелкие штрихи текстуры.

Один высокий порог — контуры **рвутся** на участках с меньшим контрастом.

Все пять шагов — тонкие, в основном непрерывные контуры объектов.

Параметры: σ и пороги



Строки — σ, столбцы — пороги $T_{\text{low}}/T_{\text{high}}$ (яркость в $[0,1]$, `skimage.feature.canny`).

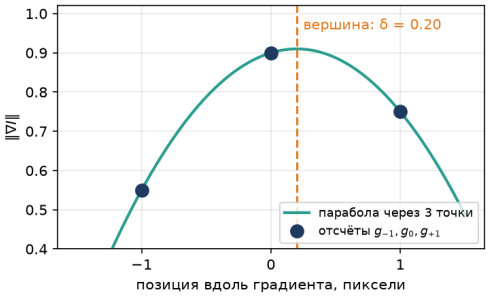
σ **задаёт масштаб**: при $\sigma = 1$ видна трава и кирпичи, при $\sigma = 4$ — только силуэт и штатив, углы скруглены.

Пороги задают «сколько»: выше — меньше границ, но длинные контуры сохраняются благодаря гистерезису.

Практические правила

- $T_{\text{high}} : T_{\text{low}} \approx 2{:}1$–$3{:}1$ (совет Кэнни);
- пороги удобно задавать **квантилями** модуля градиента: например, T_{high} — 90-й процентиль, T_{low} — 70-й;
- сначала подобрать σ под размер объектов, потом пороги.

Субпиксельная локализация границы



Пример: $g_{-1} = 0.55$, $g_0 = 0.90$, $g_{+1} = 0.75$ — правый сосед больше левого, вершина сдвинута вправо на 0.2 пикселя.

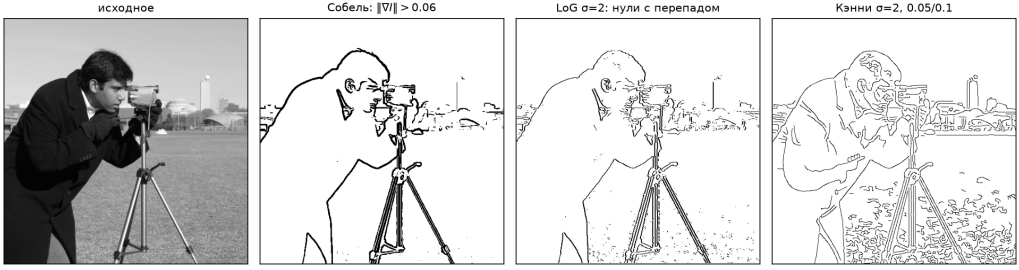
Зачем. NMS даёт границу с точностью до пикселя. Для измерений (размер детали, калибровка камеры, лекция 7) нужно точнее — **доли пикселя**.

$$\delta = \frac{g_{-1} - g_{+1}}{2\left(g_{-1} - 2g_0 + g_{+1}\right)}$$

- g_{-1}, g_0, g_{+1} — модуль градиента в пикселе-максимуме и двух соседях вдоль θ;
- $\delta \in [-\frac{1}{2}, \frac{1}{2}]$ — сдвиг вершины параболы $p(t) = at^2 + bt + g_0$ через эти три точки: $a = \frac{g_{-1}+g_{+1}}{2} - g_0$, $b = \frac{g_{+1}-g_{-1}}{2}$, вершина $t^* = -\frac{b}{2a}$;
- граница — в точке $p + \delta\,(\cos\theta, \sin\theta)$.

Типичная точность на чётких краях — 0.05–0.1 пикселя.

Собель, LoG и Кэнни на одном снимке



Порог по модулю Собеля: толстые полосы, высокий порог рвёт слабые контуры, низкий пропускает траву.

Нули LoG: тонкие и замкнутые, но **петли** на плавных перепадах и скруглённые углы.

Кэнни: тонкие, длинные и в основном непрерывные — поэтому это **стандарт** уже 40 лет.

Листинг: Кэнни

```python
import cv2
from skimage import feature

# OpenCV: uint8, свой шаг 1 - вручную
I8 = cv2.imread("cam.png", 0)
B = cv2.GaussianBlur(I8, (0, 0), 2.0)
E = cv2.Canny(B, 50, 150, L2gradient=True)
# E: 255 - граница, 0 - нет

# skimage: float, сглаживание внутри,
# пороги - квантили модуля градиента
E2 = feature.canny(I, sigma=2.0,
                   low_threshold=0.7,
                   high_threshold=0.9,
                   use_quantiles=True)
```

- `cv2.Canny` **не сглаживает** сам: только Собель 3×3. Шаг 1 делаем `GaussianBlur`.
- Пороги OpenCV — в единицах **ненормированного** Собеля (в 8 раз больше наклона), для `uint8`.
- `L2gradient=True` — модуль $\sqrt{I_x^2 + I_y^2}$; по умолчанию $|I_x| + |I_y|$ (быстрее, но зависит от направления).
- `use_quantiles`: 0.9 = «границей могут быть самые сильные 10 % пикселей».

Часть 6

Преобразование Хафа

О чём эта часть

Определение

Преобразование Хафа — способ найти на карте границ фигуры заданного вида (прямые, окружности): каждый пиксель границы **голосует** за все фигуры, которые через него проходят; фигуры с большим числом голосов — найденные.

Пространство параметров — множество всех возможных фигур, записанных числами (для прямой — два числа). **Аккумулятор** — сетка в этом пространстве со счётчиками голосов.

Что покажем

- удобную запись прямой и голосование;
- почему метод устойчив к разрывам и шуму;
- окружности, обобщённый и вероятностный Хаф;
- проклятие размерности.

От пикселей к прямым

После Кэнни у нас **множество пикселей**, а нужны **объекты**: «на снимке 20 прямых, вот их уравнения». Трудности:

- прямых **много**, и неизвестно, какой пиксель к какой относится;
- линии **прерываются**: заслонения, слабый контраст;
- много **посторонних** пикселей: текст, текстура, шум.

Метод наименьших квадратов (МНК) годится для **одной** прямой без выбросов: один посторонний пиксель далеко от прямой сильно её перекосит.

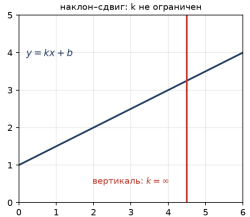
Идея Хафа (1962)

Перевернуть задачу. Не «через какие пиксели проходит прямая», а «**за какие прямые голосует пиксель**». Через один пиксель проходит бесконечно много прямых — он отдаёт голос каждой. Через n пикселей одной прямой — **она одна** получает n голосов.

Нужно решить

1) как записать «все прямые» конечной сеткой чисел; 2) как быстро раздать голоса; 3) как найти победителей.

Как записать прямую двумя числами



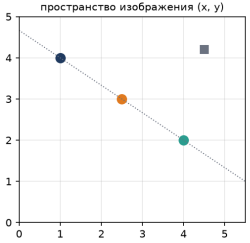


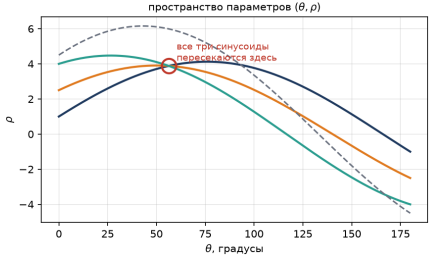
$y = kx + b$ **не годится**: для вертикальной прямой $k = \infty$, для почти вертикальных k огромно — сетку по k не построить.

$$x \cos\theta + y \sin\theta = \rho$$

- $\theta \in [0, \pi)$ — угол **нормали** к прямой (перпендикуляра из начала координат);
- ρ — расстояние со знаком от начала координат до прямой, $|\rho| \le D = \sqrt{W^2 + H^2}$ — диагональ картинки;
- оба параметра **ограничены** — сетка конечна (Duda, Hart, 1972).

Точка → синусоида





$$\rho(\theta) = x_0 \cos\theta + y_0 \sin\theta = R\cos(\theta - \varphi)$$

- (x_0, y_0) — фиксированный пиксель; $\rho(\theta)$ — все прямые через него;
- $R = \sqrt{x_0^2 + y_0^2}$, $\varphi = \operatorname{atan2}(y_0, x_0)$ — полярные координаты пикселя: получаем синусоиду.

Три точки на одной прямой дают три синусоиды, которые **пересекаются в одной точке** (θ^*, ρ^*) — это параметры прямой: здесь $\theta^* \approx 56.3°$, $\rho^* \approx 3.88$. Четвёртая точка (серая) не на прямой — её синусоида проходит мимо.

Задача «найти прямую» стала задачей «найти, где пересекается **много** синусоид».

Алгоритм голосования

Преобразование Хафа для прямых

Вход: карта границ E, шаги $\Delta\theta$, $\Delta\rho$, порог T.
Выход: список прямых (θ, ρ).

1. $A[\theta_j, \rho_i] \leftarrow 0$ для всех ячеек сетки
2. для каждого пикселя (x, y) с $E[x, y] = 1$:
   для каждого θ_j: $\rho \leftarrow x \cos\theta_j + y \sin\theta_j$;
   $A[\theta_j, \mathrm{round}(\rho/\Delta\rho)]$ += 1
3. найти **локальные максимумы** A с $A \geq T$
4. вернуть их (θ, ρ)

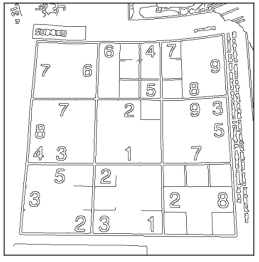
- A — аккумулятор, $N_\theta \times N_\rho$ счётчиков; обычно $\Delta\theta = 1^\circ$ ($N_\theta = 180$), $\Delta\rho = 1$ пиксель ($N_\rho = 2D$);
- T — минимальное число пикселей на прямой.

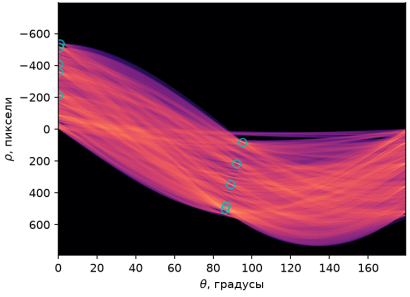
Стоимость: $O(E_\# \cdot N_\theta)$, где $E_\#$ — число пикселей границ: Кэнни сокращает его в десятки раз по сравнению со всеми пикселями.

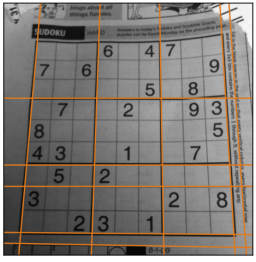
Шаг сетки — компромисс: крупный — близкие прямые сливаются, неточный угол; мелкий — голоса одной прямой **расползаются** по соседним ячейкам из-за шума, пик ниже.

Поиск максимумов — снова NMS, но в окне по (θ, ρ).

Пример: линии судоку



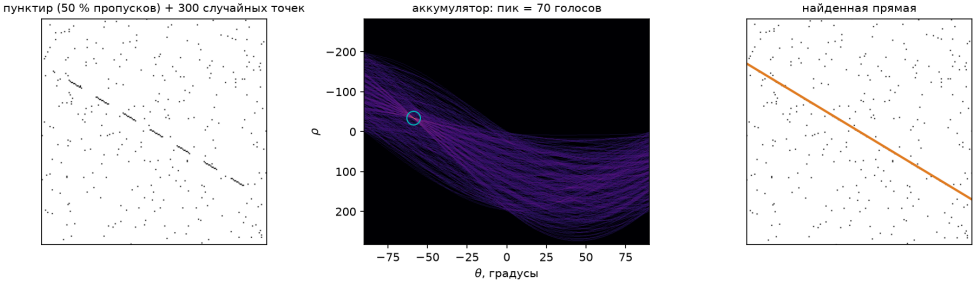




Каждый пиксель границы рисует в аккумуляторе синусоиду; там, где их пересекается много, — яркие пики. Сетка судоку — почти горизонтальные ($\theta \approx 90^\circ$) и почти вертикальные ($\theta \approx 0^\circ$ и 180°) прямые: **пики выстроены в два столбца**.

20 самых сильных пиков — 20 прямых сетки. Цифры и шум дают голоса «вразброс» и **не набирают** пика. Лист изогнут — некоторые линии сетки чуть кривые, Хаф даёт лучшую прямую через них.

Устойчивость к разрывам и шуму



Прямая — пунктир: **половина пикселей пропущена**. Плюс 300 случайных точек — больше, чем точек на прямой (86). Пик аккумулятора — 70 голосов: часть голосов прямой ушла в соседние ячейки из-за округления, но он всё равно намного выше фона.

Почему устойчиво. Пропуск пикселя лишь **уменьшает** пик, но не сдвигает его. Случайная точка голосует за свою синусоиду — она размазана по всему аккумулятору и **не создаёт** пика. МНК на этих данных провёл бы прямую «посередине облака».

Окружности: три параметра

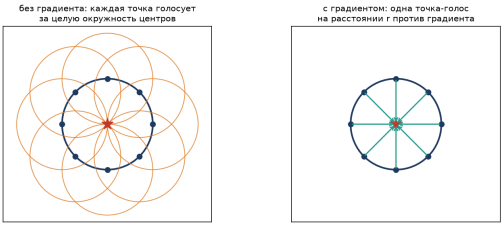
$$(x-a)^2+(y-b)^2=r^2$$

- (a,b) — центр, r — радиус: аккумулятор **трёхмерный** $A[a,b,r]$;
- при известном r пиксель (x,y) голосует за все центры на окружности радиуса r вокруг себя.

Ускорение через градиент. Нормаль к окружности направлена по градиенту, поэтому центр лежит **на прямой градиента**:

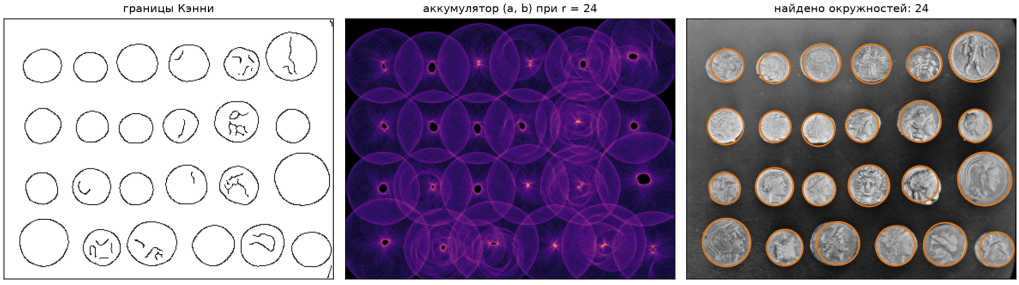
$$(a,b)=(x,y)\pm r\,(\cos\theta,\sin\theta)$$

- θ — направление градиента в пикселе; знак зависит от того, светлее круг фона или темнее (часто голосуют за оба).



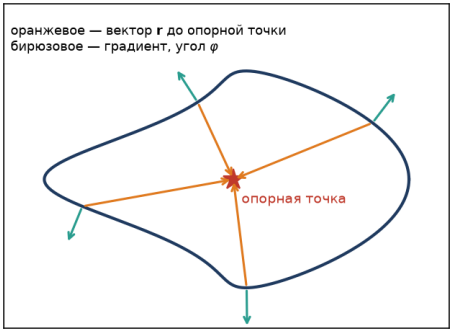
Вместо окружности голосов — **одна-две ячейки** на каждый радиус: работы в сотни раз меньше, фона в аккумуляторе тоже. Так устроен `cv2.HoughCircles` (метод `HOUGH_GRADIENT`).

Пример: монеты



Границы Кэнни ($\sigma = 3$), радиусы от 18 до 35 пикселей. В срезе аккумулятора при $r = 24$ видны яркие точки в центрах монет подходящего размера; вокруг — «кольца» голосов от пикселей других монет. Взяли 24 лучших пика по всем (a, b, r) с запретом центров ближе 20 пикселей — найдены **все 24 монеты**, хотя у некоторых граница с разрывами, а внутри — рисунок, который тоже даёт границы.

Обобщённый Хаф (Ballard, 1981)



Для фигуры **без формулы** (деталь, логотип) запоминаем по образцу, где опорная точка относительно каждого пикселя границы.

Алгоритм

Обучение (по образцу): для каждого пикселя границы p с направлением градиента φ записать вектор $\mathrm{r} = c - p$ в строку $R[\varphi]$ таблицы.

Поиск: для каждого пикселя границы q с направлением φ и каждого $\mathrm{r} \in R[\varphi]$: $A[q + \mathrm{r}] \mathrel{+}= 1$. Пик A — положение фигуры.

- c — опорная точка (например, центр образца);
- R — **R-таблица**: направление градиента $\to$ список векторов до c.

Поворот и масштаб фигуры — ещё два параметра: аккумулятор 4D. Это прямой предок голосования в детекторах объектов (Implicit Shape Model, 2004).

Вероятностный Хаф: отрезки



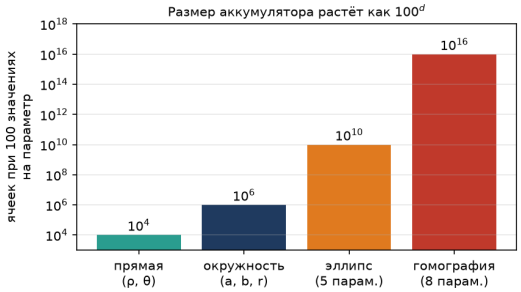




Классический Хаф даёт **бесконечные** прямые: не видно, где отрезок начинается и кончается, параллельные окна склеиваются в одну линию через весь кадр.

Прогрессивный вероятностный Хаф (Matas и др., 2000; `cv2.HoughLinesP`): голосуют **случайные** пиксели по одному; как только ячейка набрала порог, вдоль её прямой ищется отрезок (с допуском на разрывы `maxLineGap`), его пиксели **убираются** из голосования. Быстрее и сразу выдаёт концы отрезков.

Проклятие размерности



$$\#\text{ячеек} = \prod_{j=1}^{d} N_j \approx N^d$$

- d — число параметров фигуры, N_j — число значений j-го параметра в сетке.

Прямая — 10^4 ячеек, окружность — 10^6, эллипс — 10^{10}: память и время **растут экспоненциально**, а голоса размазываются по огромному числу ячеек.

Что делают

Сокращают параметры за счёт градиента (окружности); ищут поэтапно (сначала центр, потом радиус); для многих параметров — **RANSAC** (лекция 7): гипотезы по минимальной выборке вместо полной сетки.

Листинг: Хаф в OpenCV


```
E = cv2.Canny(B, 50, 150)

# прямые: массив (rho, theta)
L = cv2.HoughLines(E, rho=1,
                   theta=np.pi / 180,
                   threshold=150)

# отрезки: массив (x1, y1, x2, y2)
S = cv2.HoughLinesP(E, 1, np.pi / 180, 60,
                    minLineLength=40,
                    maxLineGap=5)

# окружности: (a, b, r), Кэнни внутри
C = cv2.HoughCircles(B, cv2.HOUGH_GRADIENT,
                     dp=1, minDist=20,
                     param1=150, param2=30,
                     minRadius=18, maxRadius=36)
```

- `rho`, `theta` — шаги сетки $\Delta\rho$, $\Delta\theta$; `threshold` — порог голосов T.
- `minLineLength` — отбросить короткие отрезки; `maxLineGap` — склеить отрезки с разрывом до 5 пикселей.
- `HoughCircles` принимает **сглаженное изображение**, а не границы: `param1` — верхний порог его Кэнни, `param2` — порог голосов за центр.
- Результат `None`, если ничего не найдено — проверять.

Часть 7

Пример: выравнивание скана документа

Deskew: угол наклона строк

Задача. Скан или фото страницы повёрнуто на несколько градусов. Для распознавания текста (OCR) и сравнения с шаблоном документа (KYC) строки должны быть горизонтальны.

Идея. Строки текста — много параллельных «прямых». Найдём угол θ, при котором голоса Хафа **собираются в самые высокие пики**.

$$\text{score}(\theta) = \sum_{\rho} A[\theta, \rho]^2$$

- $A[\theta, \rho]$ — аккумулятор по тёмным пикселям текста;
- без квадрата сумма по ρ одинакова для всех θ: **каждый пиксель голосует ровно один раз** в каждом столбце θ.

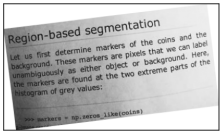
Алгоритм

1. бинаризовать (адаптивный порог или Оцу, лекция 2), текст $= 1$
2. аккумулятор Хафа только для $\theta \in [60^\circ, 120^\circ]$, шаг 0.1°
3. $\theta^* = \arg\max_\theta \text{score}(\theta)$
4. наклон строк $\alpha = \theta^* - 90^\circ$; повернуть изображение на $-\alpha$

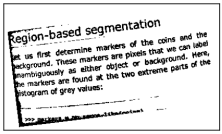
Квадрат поощряет «острые» распределения: когда угол верный, голоса строки попадают **в одну ячейку**, а не в десять соседних. Тот же смысл у дисперсии проекционного профиля.

Deskew: результат

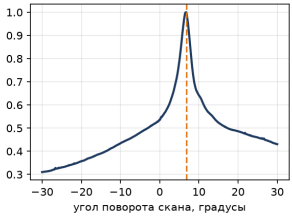
скан повёрнут на 7°



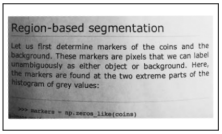
пиксели текста (бинаризация)



«сила» угла: пик при 6.8°



после поворота на найденный угол



Страницу повернули на 7°; найден угол 6.8°. Разница 0.2° — это наклон **самого исходного** снимка: на неповёрнутой странице метод находит $-0.2°$. Пик острый: ошибка в 1° уже заметно снижает score.

```
from skimage.transform import hough_line
from skimage.transform import rotate

th = np.deg2rad(np.arange(60, 120, 0.1))
A, _, _ = hough_line(text, theta=th)
score = (A.astype(float) ** 2).sum(0)
tilt = np.rad2deg(th[score.argmax()]) - 90
fixed = rotate(I, tilt, cval=1.0)
```

Мост к следующему семестру

Где эта идея живёт в глубоком обучении

- **Первые слои CNN** после обучения почти всегда содержат фильтры, похожие на **производные гауссиана и Габора** разных ориентаций (Krizhevsky и др., 2012; Zeiler, Fergus, 2014) — сеть сама «изобретает» Собеля.
- **HED** (Xie, Tu, 2015) учит карту границ по разметке людей, с выходами на нескольких масштабах — как несколько σ; после сети по-прежнему делают **NMS** из Кэнни.
- **Deep Hough Transform** (Zhao и др., 2021) — голосование в (θ, ρ) прямо по признакам сети для поиска «смысловых» линий.



Банк фильтров построен вручную по формулам этой лекции.

Подробно — в курсе «Глубокое обучение в задачах компьютерного зрения». Здесь только связь, без деталей.

Итоги лекции

1. Граница — место быстрого изменения яркости; детектор не знает её физической причины.
2. Производная на пикселях — конечная разность; центральная точнее ($O(h^2)$). Градиент: модуль — сила границы, θ — направление роста яркости, граница перпендикулярна θ.
3. Дифференцирование усиливает высокие частоты, а значит, **шум**: сначала сглаживать. Собель $= [1, 2, 1]^\top$ (сглаживание вдоль) $\times$ $[-1, 0, 1]$ (разность поперёк).
4. Производная гауссиана ∂G_σ сглаживает и дифференцирует одной свёрткой; σ — **масштаб** границ: устойчивость против локализации.
5. Нули LoG — замкнутые контуры (Марр–Хилдрет); DoG $\approx (k-1)\sigma^2$ LoG.
6. Кэнни: Гаусс $\to$ градиент $\to$ **NMS** (толщина 1) $\to$ два порога $\to$ **гистерезис** (связность).
7. Хаф: пиксели голосуют за параметры; прямая $x\cos\theta + y\sin\theta = \rho$; устойчив к разрывам и шуму, но аккумулятор растёт как N^d.

Литература

Основная

Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., гл. 7.2 (Edges and contours), 7.4 (Lines and vanishing points) — `https://szeliski.org/Book/`.
Gonzalez, Woods, *Digital Image Processing*, 4th ed., гл. 10.2 (Point, line, and edge detection).

Первоисточники

Canny, *A computational approach to edge detection*, IEEE TPAMI, 1986.
Marr, Hildreth, *Theory of edge detection*, Proc. R. Soc. Lond. B, 1980.
Duda, Hart, *Use of the Hough transformation to detect lines and curves in pictures*, CACM, 1972.
Ballard, *Generalizing the Hough transform to detect arbitrary shapes*, Pattern Recognition, 1981.
Matas, Galambos, Kittler, *Robust detection of lines using the progressive probabilistic Hough transform*, CVIU, 2000.
Изображения судоку и здания — OpenCV `samples/data` (лицензия Apache 2.0); остальное — `skimage.data`, иллюстрации построены скриптом `scripts/make_figs_L05.py`.

Вопросы для самопроверки

1. Какие бывают типы границ и какие физические причины их создают?
2. Запишите разность вперёд и центральную разность. Какая точнее и почему (ряд Тейлора)?
3. Что такое градиент, его модуль и направление? Как направление градиента связано с направлением границы?
4. Почему дифференцирование усиливает шум? Объясните через дисперсию шума.
5. Покажите, что ядро Собеля разделимо. Что делает каждый множитель?
6. Почему $\partial_x(G_\sigma * I) = (\partial_x G_\sigma) * I$? Как σ влияет на шум и на локализацию?
7. Запишите LoG. Почему граница — ноль второй производной? Как DoG приближает LoG?
8. Опишите шаги Кэнни. Зачем нужны NMS и гистерезис, что будет без них?
9. Как записать прямую для Хафа и почему не $y = kx + b$? Во что переходит точка изображения?
10. Почему Хаф устойчив к разрывам и шуму? Как ускорить поиск окружностей и в чём проклятие размерности?